

```
- alert: CronJobExecutionTimeExceeded
  expr: |
    max_over_time(kube_job_completion_time_seconds{job="my-cronjob"}[5m]) > 300
  for: 1m
  labels:
    severity: warning
  annotations:
    summary: "CronJob {{ $labels.job }} execution time exceeded allowed SLA"
    description: "CronJob {{ $labels.job }} took {{ humanizeDuration $value }} to complete"
```

Figure 4: Creating a job

```
0 5 * * 1-5 [ $(date +%w) -ne 2 ]
```

This example runs the job every week day at 5:00 a.m. (Monday to Friday) but skips the job on Tuesdays. The `$(date +%w)` command gets the day of the week, where Monday is 1 and Sunday is 7. The expression `-ne 2` means ‘not equal to 2’, which excludes Tuesdays.

- e. To run a job when a specific file or job completes, you can use a file trigger or job trigger. Here’s an example using a file trigger:

```
0 5 * * * /path/to/command && test -e /path/to/file
```

The first command will run the job every day at 5:00 a.m. If it succeeds, the test command will check if the specified file exists. If the file exists, the second command will run. If the file doesn’t exist, the second command won’t run.

Complex examples of job scheduling

Here’s an example of how to create an alert for a CronJob that runs longer than 5 minutes.

1. Create a Kubernetes job completion event rule in Prometheus as shown in Figure 4. This rule uses the `kube_job_completion_time_seconds` metric provided by `kube-state-metrics exporter` to track the completion time of the CronJob. The `max_over_time` function calculates the maximum completion time over the last 5 minutes and triggers an alert if it exceeds that.
2. Configure Prometheus to send alerts to a notification system such as Slack or PagerDuty.
3. Apply the Prometheus alert rule to your Kubernetes cluster using a `PrometheusRule` object.

To create a job that depends on another job, specify the name of the job in the `spec.dependencies` field (refer Figure 5).

Containerisation has revolutionised the way applications are developed, deployed, and managed. Kubernetes CronJob is a powerful tool for scheduling and running periodic jobs in a containerised environment, enabling users to take full advantage of the benefits of containerisation. **END** 🐧

```
apiVersion: batch/v1
kind: Job
metadata:
  name: job-a
spec:
  template:
    spec:
      containers:
        - name: job-a
          image: my-image
          restartPolicy: OnFailure
      backoffLimit: 3
---
apiVersion: batch/v1
kind: Job
metadata:
  name: job-b
spec:
  template:
    spec:
      containers:
        - name: job-b
          image: my-image
          restartPolicy: OnFailure
      backoffLimit: 3
      dependencies:
        - name: job-a
```

Figure 5: Creating a job that depends on another job

Acknowledgements

I would like to thank Balakrishnan Srinivasan (CTO - hybrid cloud application modernization services, IBM India) for encouraging me to write this article, Gopalakrishna U. (associate partner, IBM India) for giving me the opportunity to gain real-time experience on this subject and Harsh Mehta (delivery manager, IBM India) who helped me release this article for publishing.

By: Venkateswaran N.

The author is a lead architect, containerization and modernization, AWS practice, IBM India.